CNSQ DESKTOP APPLICATION SPRINT

1-Week Technical Feasibility & Implementation Roadmap (Tauri v2)

CONNECT NIGERIA

Sprint Window: 5 Days

EXECUTIVE SUMMARY

- Objective: Package existing CNSQ Web App into a native desktop application (Windows & macOS).
- Zero Backend Rewrite: Preserves 100% of the existing Next.js frontend and NestJS backend APIs.
- Technology: Tauri v2 - delivers ultra-lightweight installers (~15 MB) and minimal RAM usage (~40 MB), avoiding the heavy memory and resource overhead of traditional frameworks like Electron.

5-DAY SPRINT SCHEDULE & MILESTONES

DAY	FOCUS AREA	KEY DELIVERABLES & OBJECTIVES
Day 1 (Mon)	**Environment & POC**	- Initialize Tauri v2 framework inside the codebase. - Compile and launch local native desktop window running CNSQ. - Milestone: Verified local desktop window rendering CNSQ.
Day 2 (Tue)	**Auth & API Integration**	- Validate authentication flows (JWT tokens, sessions, secure storage). - Verify all API endpoints, ticketing, and chat routes communicate smoothly. - Configure external links to open cleanly in default system browser.
Day 3 (Wed)	**Desktop Branding & UI**	- Embed official CNSQ icons (Windows .ico and macOS .icns). - Implement native window handling & minimize-to-system-tray behavior. - Configure auto-launch on system startup capability.
Day 4 (Thu)	**Native OS Features**	- Implement native OS desktop push notifications (tickets & chat). - Conduct memory/performance profiling (target: under 80 MB RAM). - Execute cross-platform build test (Windows & macOS).
Day 5 (Fri)	**Packaging & Demo**	- Generate production installer packages (.exe/.msi & .dmg). - Perform internal QA validation on fresh test machines. - Milestone: Deliver installers & present live demo to management.

CORE BUSINESS VALUE

Independent Native Software:
Real installable app (.exe / .dmg), not a PWA.

Zero Interruption to Web Users:
Web and Desktop share the same codebase.

System Presence:
Icon on desktop, dock, system tray, & startup.

Speed & Efficiency:
Under 15 MB installer size, under 80 MB RAM.

POST-SPRINT ROADMAP (PHASE 2)

Code Signing & Trust:
Apple Developer & Windows Authenticode certs to eliminate 'Unknown Publisher' security warnings.

Background Auto-Updater:
Seamlessly pushes future releases to employees.

CI/CD Build Automation:
GitHub Actions workflow to compile binaries.

Granular Offline Caching:
Local caching for offline ticket drafting.